

Agentix

Autonomous Agentic Issue Resolution & Self-Healing Software Platform

Topic: Autonomous Multi-Agent Software Engineering Systems

Core Stack: FastAPI (Backend Orchestrator) • React (Telemetry & Human Intake) • Local Sandbox Runner

Architecture Pattern: State Machine Agent Orchestration with Policy-Based Human Governance

Audience: Computer Science Class / Technical Seminar / Engineering Viva

1. Classroom Presentation Script (3-Minute Oral Pitch)

If you are called up to explain this project to your professor and peers, read or speak this exact script:

🔪 HOW TO SPEAK ABOUT THIS PROJECT IN FRONT OF YOUR CLASS:

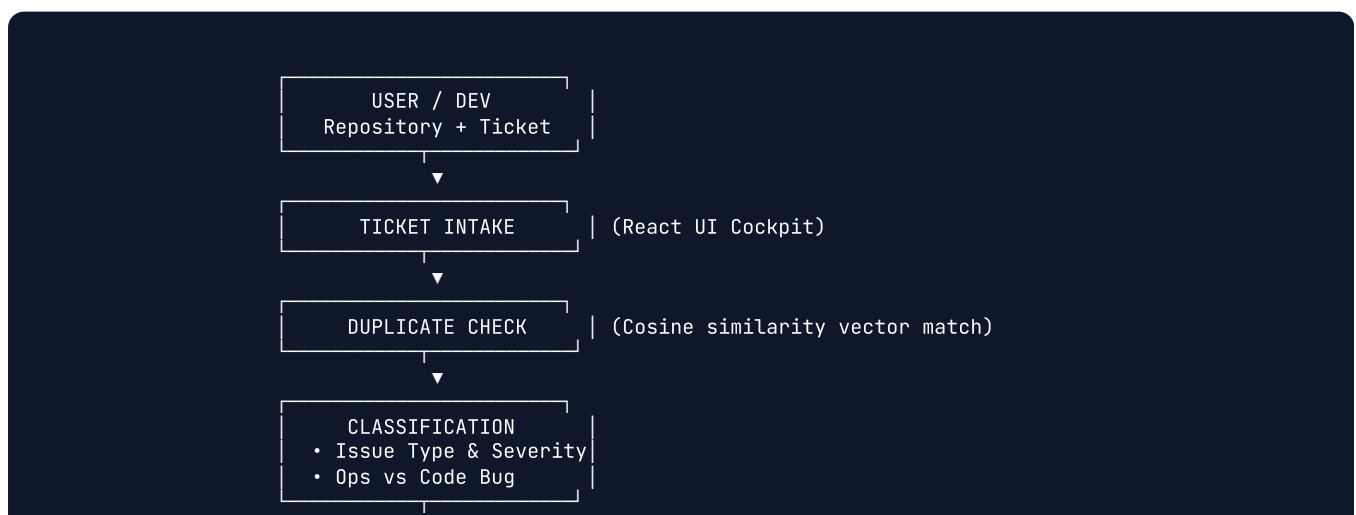
"Good morning everyone. Today I'm presenting **Agentix**, an autonomous AI-driven issue resolution platform designed to solve one of the biggest bottlenecks in modern software development: ticket triage, root cause diagnosis, and bug fixing.

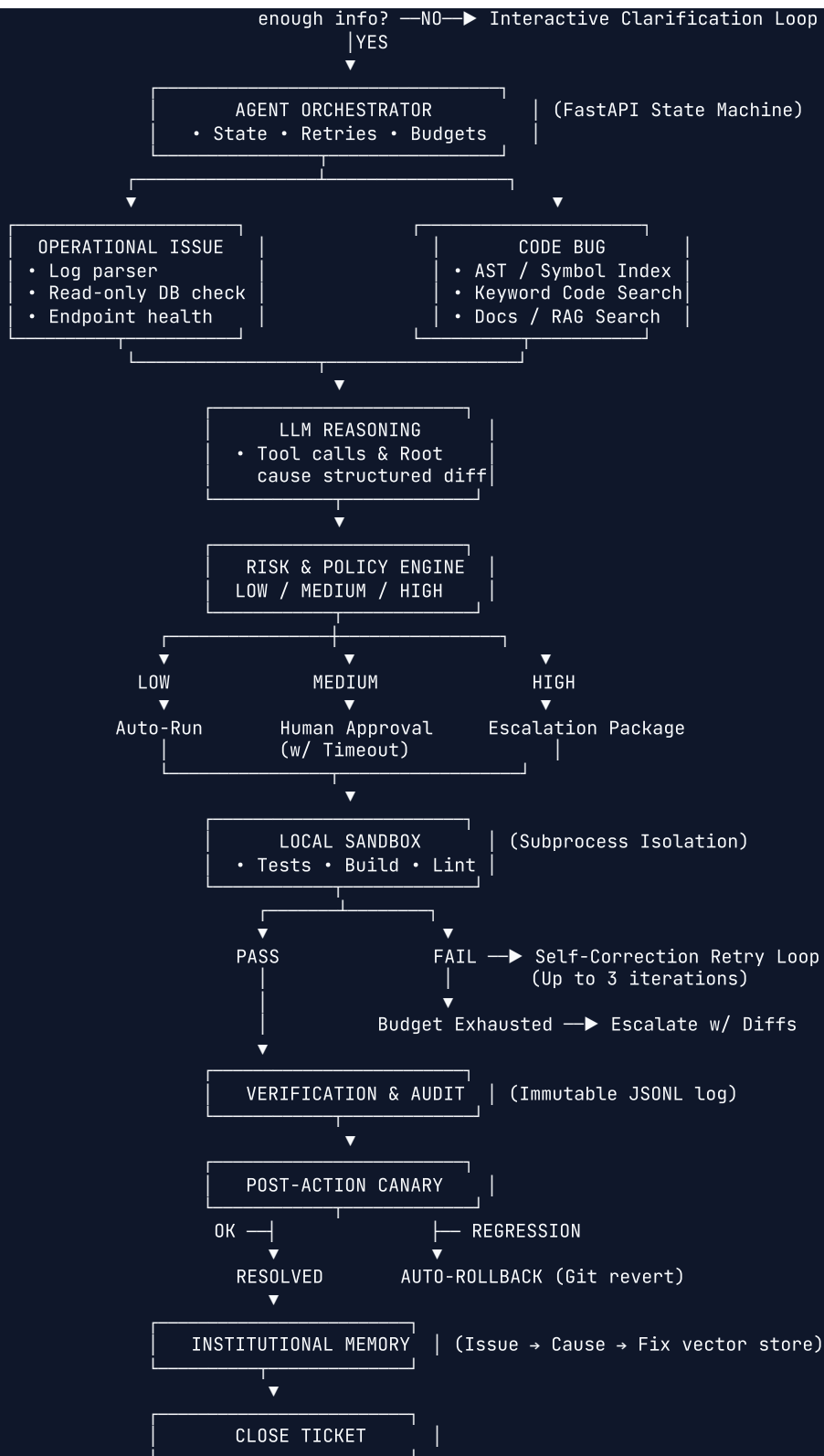
In a typical tech company, when an issue arises, engineers spend hours reading logs, checking for duplicate tickets, hunting through repositories, generating patches, and verifying that the fix doesn't break production.

Agentix automates this entire lifecycle as a multi-agent system:

1. First, a ticket enters our React UI, where vector embeddings instantly detect and merge duplicates.
2. Next, a classifier categorizes the issue: if it's operational, an Investigation Agent parses logs and checks system health; if it's a code bug, a Repository Agent inspects the codebase using Abstract Syntax Trees (AST).
3. If information is missing, the system pauses and asks the user clarifying questions.
4. An LLM reasoning loop determines the root cause and generates a git diff patch.
5. Crucially, we enforce safety: a **Risk Engine** tiers actions into Low, Medium, and High risk. High-risk issues require human sign-off with a full evidence package.
6. Code fixes are automatically validated in a safe local sandbox running builds and unit tests with a self-correcting retry loop.
7. Finally, after passing an observation canary window, the fix is permanently stored into Institutional Memory so the exact same problem never needs to be solved from scratch again."

2. System Architecture Flowchart





3. Step-by-Step Technical Execution Walkthrough

Step 01 Ticket Intake (React UI)

React REST API Pydantic

A software engineer or customer submits a problem report containing:

- **Title & Description:** What happened versus expected behavior.
- **Repository Link / Path:** Target source code repository.
- **Error Logs & Stack Traces:** Raw logs copied from the terminal or production monitoring.

Step 02 Duplicate Detection Engine

Vector Embeddings Cosine Similarity

Before launching expensive agent workflows, the ticket's text is converted into vector embeddings. If cosine similarity with an existing active ticket is > 0.88 , the system marks it as a duplicate, merges the reporters into the parent ticket, and prevents redundant compute.

Step 03 Triage & Route Classification

LLM Classification Deterministic Heuristics

The triage engine analyzes the ticket and determines:

- **Issue Type:** Crash, Performance Regression, UI Flaw, Configuration Mismatch.
- **Severity:** P0 (Critical Outage), P1 (Major), P2 (Moderate), P3 (Minor).
- **Route:** Operational Issue (infra/DB/network) vs Code Bug (source code fault).

Step 04 Interactive Clarification Loop

Human-in-the-Loop WebSockets

If the user provided vague text (e.g., *"The app crashes when I click submit"* without stack traces or environment info), the agent does NOT guess. It transitions to `NEEDS_CLARIFICATION`, generates precise questions, and waits for user input before proceeding.

Step 05 Agent Orchestrator (Control Plane)

FastAPI Finite State Machine Budget Controller

The central orchestrator manages state transitions and enforces strict financial & safety budgets:

- **Max Tool Calls:** Caps agent tool calls to 15 per ticket to prevent infinite loops.
- **Time Budget:** Max 300 seconds total execution timeout.
- **Retry Limits:** Max 3 sandbox code rebuild attempts.

Step 06 Specialized Autonomous Agents

Branch A: Operational Agent

Branch B: Repo Agent

- **Operational Agent:** Queries local log files, executes read-only SQL diagnostics (rejects any DROP, INSERT, UPDATE), and performs HTTP health check pings.
- **Repository Comprehension Agent:** Scans the target folder, parses code into an **Abstract Syntax Tree (AST)** to map classes, functions, and API routes, and performs keyword searches against stack trace terms.

Step 07 LLM Reasoning & Root Cause Analysis

Structured Output

Unified Diff Format

The reasoning model synthesizes evidence from the specialized agents and produces a structured verdict:

- **Root Cause Explanation:** Exactly why the bug occurs.
- **Target File & Line Numbers:** Exact location of the error.
- **Unified Diff Patch:** Ready-to-apply + / - git diff code modification.

Step 08 Risk & Policy Engine (Guardrail Gateway)

Policy Engine

Human Governance

Risk Tier	Trigger Conditions	Enforced Action
LOW	Docstrings, typos, read-only operational diagnostics	Auto-executed immediately, recorded in audit log.
MEDIUM	Standard logic fixes, refactors, utility edits	Pauses for Human Approval in UI. Auto-escalates on timeout.
HIGH	Auth, passwords, database schema, payment modules	Auto-escalates with a bundled <i>Evidence Package</i> for human review.

Step 09 Local Isolated Sandbox & Verification

Subprocess Isolation

Pytest / Linter

Approved patches are applied to a temporary working branch in a local sandbox:

1. **Syntax & Linting:** Ensures zero syntax errors or formatting breaks.
2. **Build Verification:** Confirms the application still builds clean.

3. **Automated Test Suite:** Runs existing unit and integration tests.

Step 10 Self-Correcting Retry Loop

Feedback Loop

Iterative Refinement

If the sandbox tests fail, the compiler/test error trace is piped back to the reasoning agent. The agent examines why its patch failed, modifies the code, and re-tests (up to 3 maximum attempts). If all attempts fail, it escalates with the full diff history.

Step 11 Immutable Audit Trail & Telemetry

JSONL Ledger

Auditability

Every single step—user inputs, LLM prompt tokens, tool invocations, bash outputs, and approvals—is written to an append-only JSONL ledger. This provides 100% compliance and forensic explainability.

Step 12 & 13 Post-Action Canary & Automated Rollback

Canary Observer

Git Revert

Once a fix is applied, the system enters an observation window:

- **No new errors:** The ticket is marked RESOLVED.
- **New regression detected:** The system automatically issues a `git revert` to roll back the patch immediately and flags the ticket for senior engineering escalation.

Step 14 & 15 Institutional Memory & Ticket Closure

Vector Memory

Feedback Learning

When an issue is successfully resolved, the tuple of (Issue Description, Root Cause, Solution Patch, Verification Logs) is indexed into **Institutional Memory**. The next time a similar bug surfaces anywhere in the company, the system recalls the exact solution instantly.

4. Anticipated Viva / Professor Questions & Strong Answers

Q1: Why do you need both an Operational Agent and a Repository Comprehension Agent?

Answer: Software issues fall into two distinct domains: infrastructure/operational issues (e.g., 502 Bad Gateway, high memory usage, database connection pools) and source code logic bugs (e.g., null pointer exceptions, incorrect algorithms). Specialized agents use specialized tools: log/DB tools for operations, and AST/symbol indexers for source code.

Q2: How do you prevent the AI from generating dangerous code or deleting the database?

Answer: Through our three-tiered **Risk & Policy Engine**: read-only database connections, keyword safety blockers (blocking words like DROP, DELETE, AUTH), and mandatory human approval for medium-to-high risk operations before anything touches execution.

Q3: How do you prevent hallucinated fixes?

Answer: The **Local Sandbox Runner** acts as an objective ground-truth gate. The agent's patch is never accepted based on LLM confidence alone; it must pass static syntax analysis, build compilation, and automated test execution.